

QUANTISATION AND PRECISION

Adrian Jackson

a.jackson@epcc.ed.ac.uk

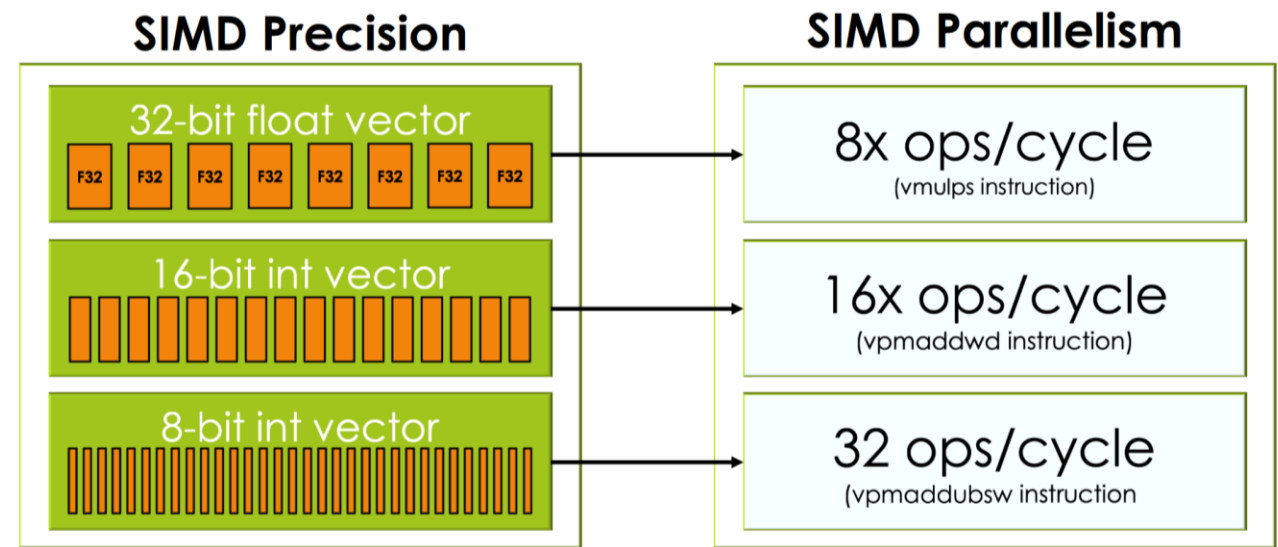


THE UNIVERSITY
of EDINBURGH



Quantisation

- Data type used in arithmetic affects performance
 - Vectorisation can use range of data types, impacts through put/parallelisation of given workload
- Number of bits of a datatype key to what data it can store
 - Range of numbers
 - Accuracy of numbers
- Integer bits
 - Only controls ranges
- Floating point bits
 - Controls range and precision



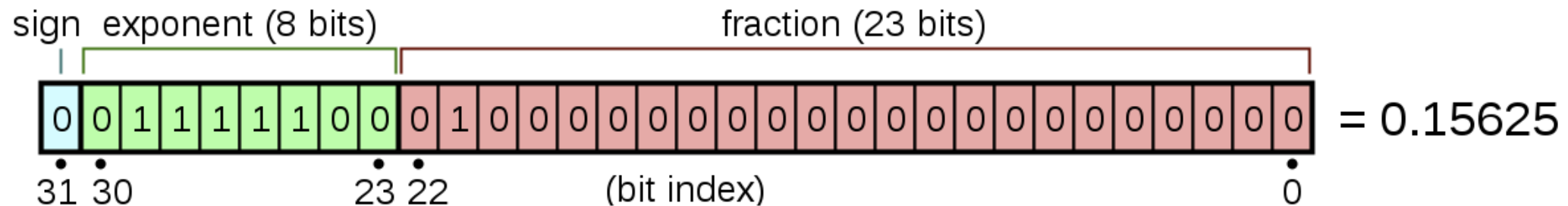
Fixed sized datatypes

- For floating point numbers (Reals)
 - A finite representation for infinite numbers
- The representation is bounded by:
 - The base used (β i.e. 2, 3, 10, etc...)
 - The range of the exponent (e)
 - The number of significands (p)
- Called floating point because changing the exponent changes where the floating point is in the final number, i.e.
 - $0.1234 \times 10^2 = 12.34$
 - $0.1234 \times 10^{-2} = 0.001234$
 - $0.1234 \times 10^4 = 1234.0$

$$12.345 = \underbrace{12345}_{\text{significand}} \times \underbrace{10^{-3}}_{\text{base}}^{\text{exponent}}$$
$$\pm(0.d_1d_2 \dots d_p)_\beta \times \beta^e$$

Fixed sized datatypes

- Useful for computing because can fix the size of your data type
 - Define β , the range of the exponent ($\pm m$), the number of significands (p)
 - Defines the number of bits needed to store that datatype
 - Assuming binary computing ($\beta = 2$) then:
 - Using 32-bits, with 8-bits for the exponent range, 23-bits for the significand precision and 1-bit for the sign () we can store:

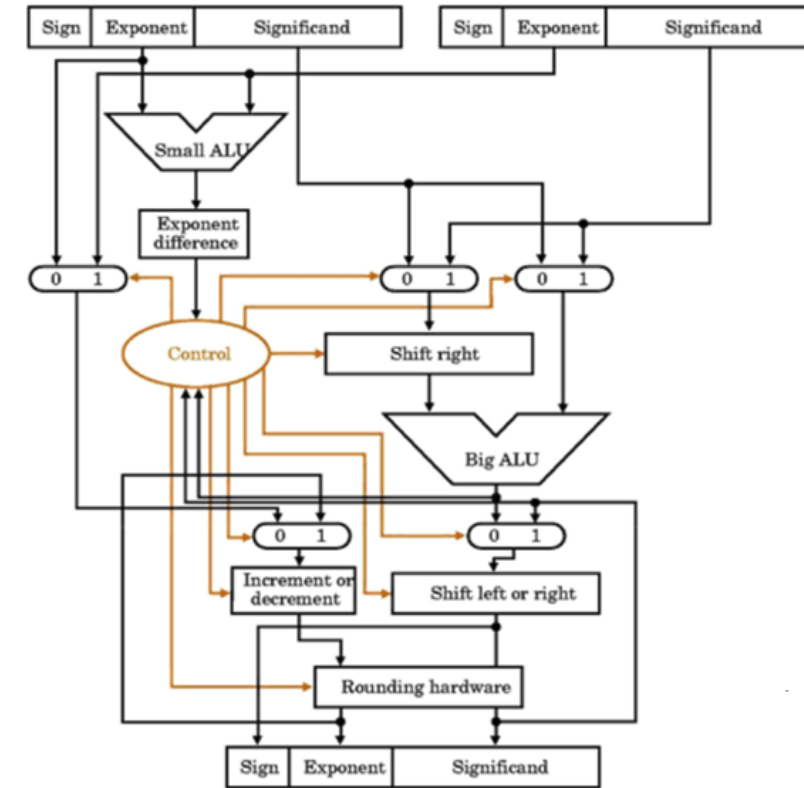


Fixed size datatypes

- Numbers smaller or larger than the fixed size can store will
 - Overflow or underflow: Computation succeeds but the number produced is not correct
- Fix size data types will be imprecise in calculations
 - Rounded to the precision of the datatype
- Lots of complexity in special operations
 - Zero representations, maths functions, etc...
- Integer datatypes are exact
 - But limited in range
 - 32-bit integer gives range of $-(2^{31})$ to $(2^{31} - 1)$ or $-2,147,483,648$ to $2,147,483,647$

Floating point arithmetic

- Maths operations can be defined algorithmically
 - Multiplication: $f_1 \times f_2 = (f_{1m} \times f_{2m}) \times 2^{(f_{1e} + f_{2e})}$
 - Division: $\frac{f_1}{f_2} = \left(\frac{f_{1m}}{f_{2m}}\right) \times 2^{(f_{1e} - f_{2e})}$
 - Separate mantissa and exponent
 - Process them separately
 - Combine them together
- Addition requires a normalisations as well
 - Make exponents same value and shifting significands to compensate
- Requires fixed sized mantissa (significand) and exponent parts



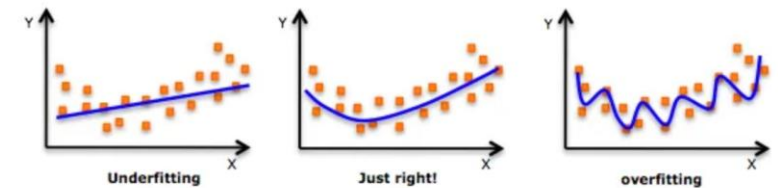
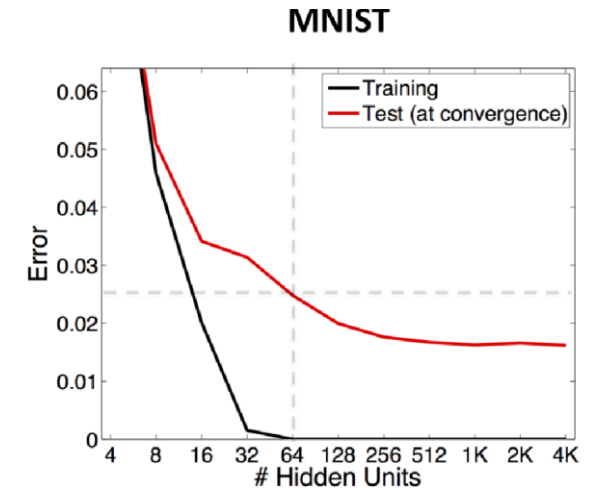
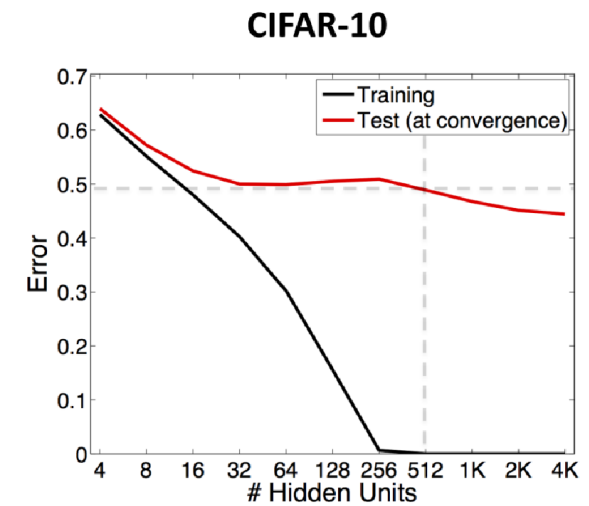
Standard floating point data types

Type	Bits	Exponent bits	Mantissa bits	Smallest size	Largest Size	Machine epsilon
Double Float*	64	11	53	$\sim 10^{-308}$	$\sim 10^{308}$	$\sim 10^{-16}$
Single Float*	32	8	24	$\sim 10^{-38}$	$\sim 10^{38}$	$\sim 10^{-7}$
Half Float*	16	5	11 (10)	$\sim 5^{-8}$	$\sim 6^4$	$\sim 10^{-4}$
BFloat16	16	8	7	$\sim 10^{-38}$	$\sim 10^{38}$	$\sim 10^{-3}$
TF32	19	8	10	$\sim 10^{-38}$	$\sim 10^{38}$	$\sim 10^{-4}$
Double Int	64	N/A	N/A	-2^{63}	$2^{63} - 1$	N/A
Single Int	32	N/A	N/A	-2^{31}	$2^{31} - 1$	N/A
Half Int	16	N/A	N/A	-32768	32767	N/A
Quarter Int	8	N/A	N/A	-128	127	N/A

*IEEE754 standard

DNN quantisation

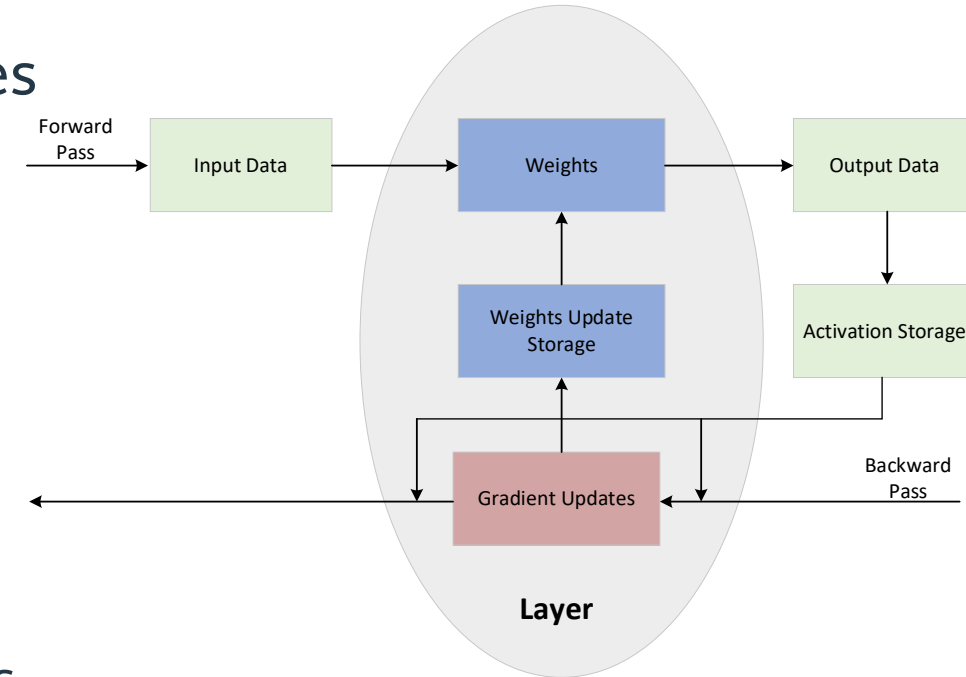
- DNN are generally significantly over parameterised
 - Trade offs between design and functionality
 - Maximising generalisability with finite datasets
- Because they are over parameterised...
 - ...they can tolerate reduced data precision in places
 - Form of information compression
- Can be done in different places
 - Network design (pre-training)
 - For inference (post-training)
- Post-training can be done more simply
 - Required precision for weights can be evaluated
 - Very small weights can be pruned/considered zero



An example of overfitting, underfitting and a model that's "just right!"

DNN data

- Can split our DNN variables into five different types
 - Input dataset(s): The data used to train, or for inference with, the DNN
 - Weights and Biases: The numbers that are trainable in the network
 - Layers: The datatypes used within individual layers to undertake tensor operations
 - Gradients: Variables used in the gradient calculations to update weights
 - Communications: Datatypes employed in communications between layers/devices
- Different parts of the DNN can benefit from different precision
 - i.e. Gradient calculation might benefit from high precisions, input data might only need low precision (i.e. pixel values)

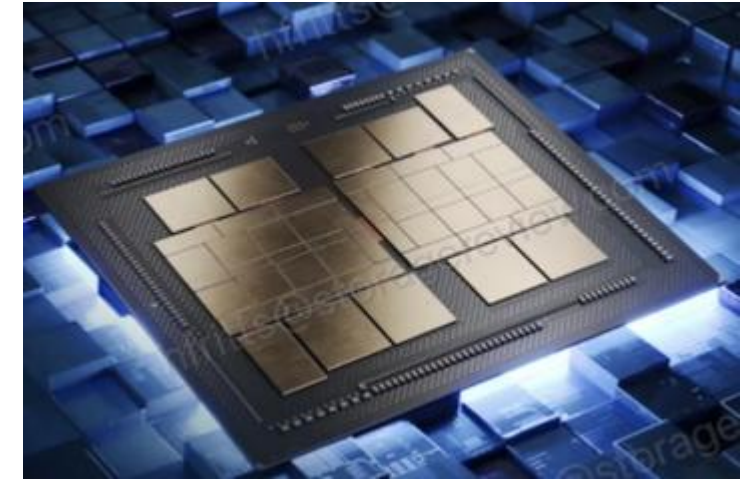
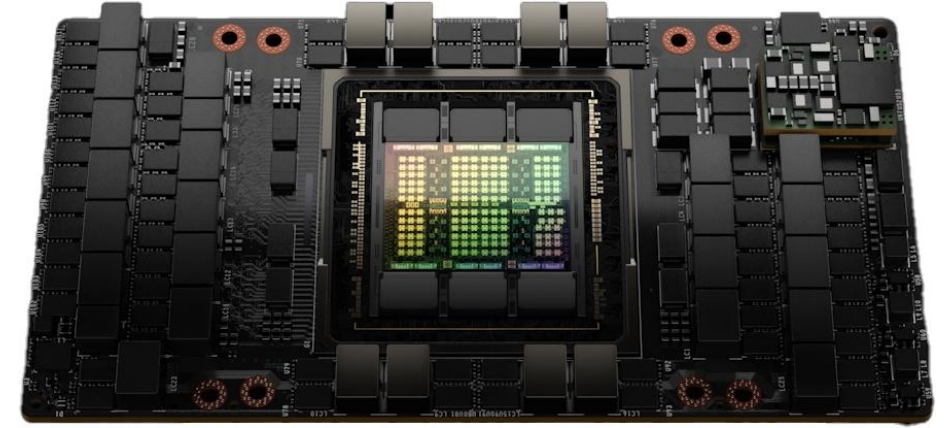


DNN data

- Can also train at higher precision than inference
 - Maintain larger datatypes at training to cope with range/precision issues
 - Convert to smaller types after trained to save space and improve inference performance
 - Form of compression (see later lecture)
- Considerations for sensitive functionality
 - SGD can benefit from unbiased rounding in reduced precision
- More complex datatypes have been suggested
 - Shared exponent floating point type
 - Lookup table exact values
- Reduced precision only helps if the hardware supports it

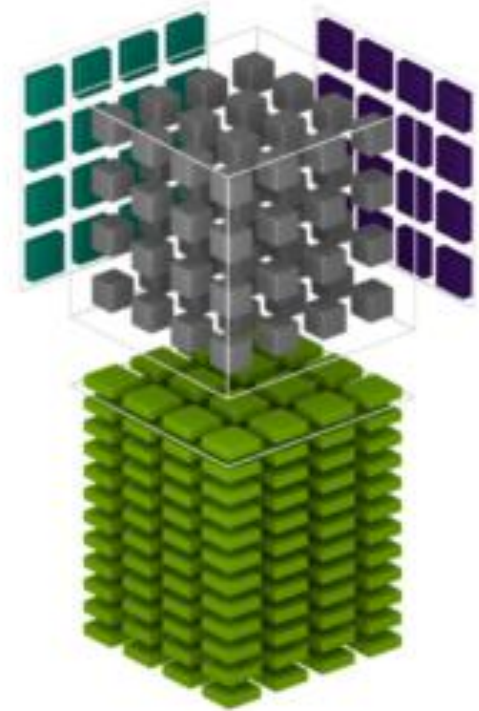
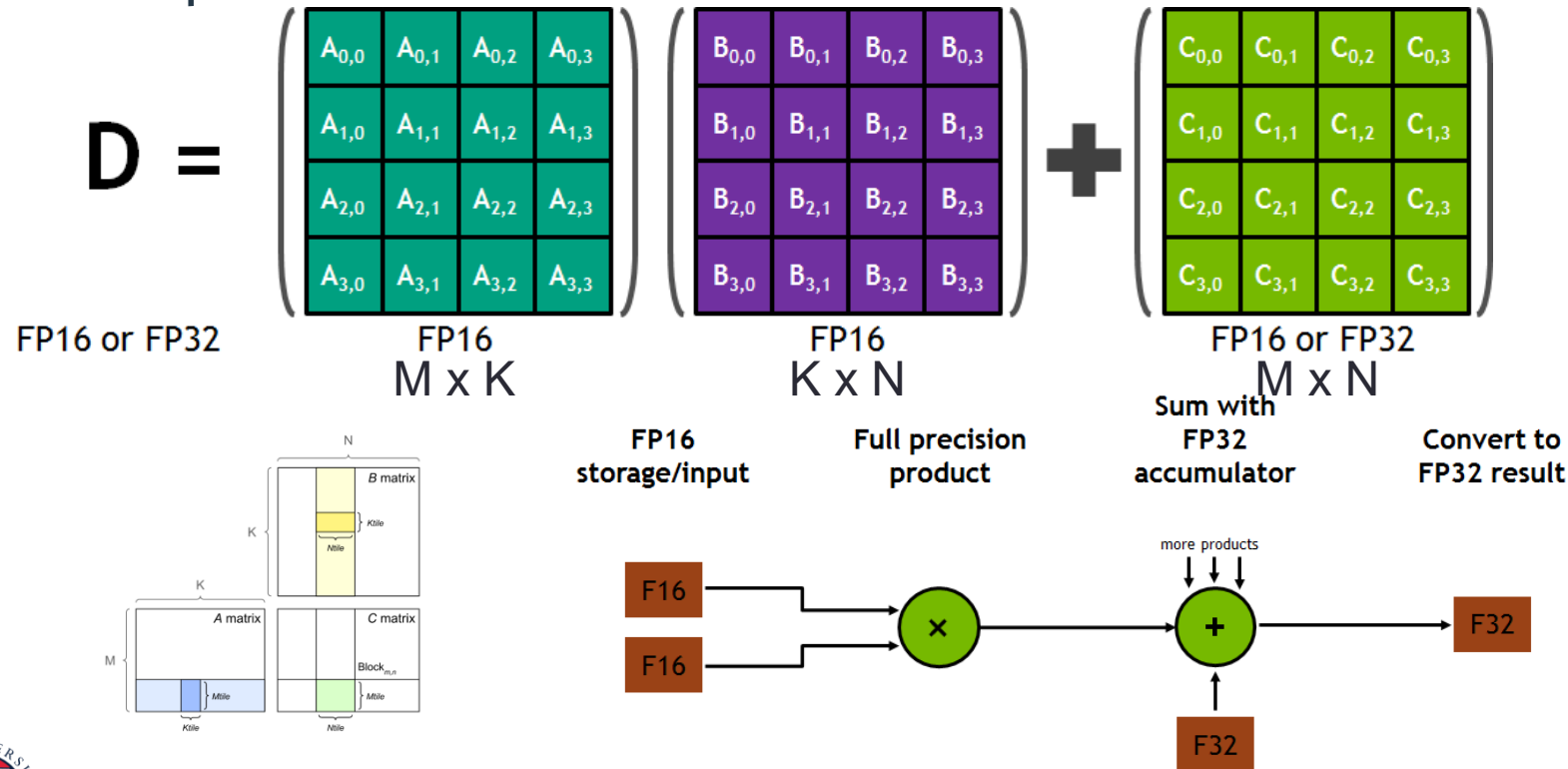
Hardware support

- Nvidia H100 GPUs
 - 8 Graphics Processing Clusters (GPCs)
 - 66 Texture Processing Clusters (TPCs):
 - 2 SM/TPC
 - 132 Streaming Multiprocessors (SMs):
 - 128 FP32 CUDA Cores/SM
 - 4 Tensor Cores/SM
 - FP8, FP16, BF16, TF32, FP64, and INT8 MMA data types are supported on the Tensor Cores
- AMD Mi300 GPUs
 - 1216 Matrix Cores (MC)
 - 16 Stream Processors (SP) per/MC
 - INT8, FP8, BF16, FP16, TF32, FP32, and FP64 supported on the Matrix Cores
 - MFMA operations equivalent to tensor core functionality
- Intel Sapphire Rapids Max CPUs
 - Up to 40 cores per processor
 - AVX512 vector units: FP16, FP32, FP64
 - Intel AMX (Advanced Matrix Extensions): BF16, Int8 functionality



Nvidia Tensor cores

- Tensor cores are matrix – matrix multiplication engines
 - This operation requires multiplication and accumulation of fixed size operations



Hardware restrictions

- Tensor Cores have fixed size requirements on the input tensors:
 - For fp16 inputs, all three dimensions (M, N, K) must be multiples of 8
 - For int8 inputs, all three dimensions must be multiples of 16
 - Convolutions also require
 - Using fp16 inputs, input and output channels must be multiples of 8
 - Using int8 inputs, input and output channels must be multiples of 16
- Example requirement for TensorFlow:
 - `tf.keras.layers.Dense(units=64)`
 - `tf.keras.layers.Conv2d(filters=48, kernel_size=7, stride=3)`
 - `tf.keras.layers.LSTM(units=64)`
 - `tf.keras.Model.fit(epochs=2, batch_size=128)`
- AMD MFMA works on range of sizes, generally divisible by 4 required
- Intel AMX works on 16x64-byte tiles
 - i.e. 16 x 64 int8 or 16 x 32 bf16

Summary

- Quantisation covers a number of topics
 - Using hardware specific datatypes
 - Exploiting over parameterised nature of a trained network
- Trade off between network design and performance
- Training and inference
 - Different requirements and approaches
- Significant performance benefits can be achieved
 - Trade off with accuracy
 - Clever design can benefit (mixed-precision)
- Mapping to hardware can be very beneficial